

Documentation d'architecture

Cette documentation servira à décrire les choix d'architecture ainsi que les spécifications de l'API REST.

1. Introduction

Ce document présente l'analyse et les propositions d'architecture réalisées sur l'application Renote, un outil de prise de notes permettant à un utilisateur de créer des notes, de leur associer des tags, et de gérer son compte. Elle est aujourd'hui implémentée sous la forme d'un monolithe Laravel/Livewire : l'intégralité de la logique (affichage, validation, accès aux données) est portée par des composants serveur, sans séparation entre le front et le back, et sans API exposée.

Cette architecture atteint aujourd'hui ses limites : elle ne permet pas de réutiliser la logique métier pour d'autres clients qu'un navigateur web classique, ce qui bloque notamment tout développement d'une application mobile, objectif désormais fixé au projet.

L'objectif de ce document est de documenter la démarche de transformation vers une architecture découplée :

- un front-end React, autonome, consommant une API HTTP,
- un back-end Laravel réorganisé selon les principes MVC et SOLID, exposant une API REST avec un contrat de réponse standardisé,
- une séparation claire des responsabilités (affichage, état, appels réseau côté front ; routage, contrôle, logique métier, accès aux données côté back), condition nécessaire pour que l'API soit réellement réutilisable par plusieurs clients.

Le document suit une trame en trois temps : l'analyse de l'architecture existante, l'analyse de l'écart entre l'existant et la cible, puis la description de l'architecture cible et la justification des choix retenus (notamment le pattern de gestion de « state » côté front).

2. Architecture de base



Point forts :

- **Peu de code par feature** : Livraison en prod rapide
- **État unifié** : Pas de problème de synchro front/back
- **Pas d'API publique** : Risque d'attaque faible
- **Un seul framework** : Onboarding rapide, apprentissage rapide
- **Mono artefact** : Déploiement Rapide

Optimise brutalement le ratio fonctionnalités livrées / effort investi pour des apps CRUD à faible complexité métier, mono-client (web only), avec une petite équipe qui maîtrise Laravel.

Points faibles :

- **Testabilité faible** : couplage direct à Eloquent, impossible de mocker sans base réelle.
- **Pas de séparation métier / infrastructure** : les règles métier sont disséminées entre validation Livewire, filtres SQL et contraintes de migration.
- **Pas d'ouverture** : aucune API, impossible de brancher un client mobile ou un front

séparé.

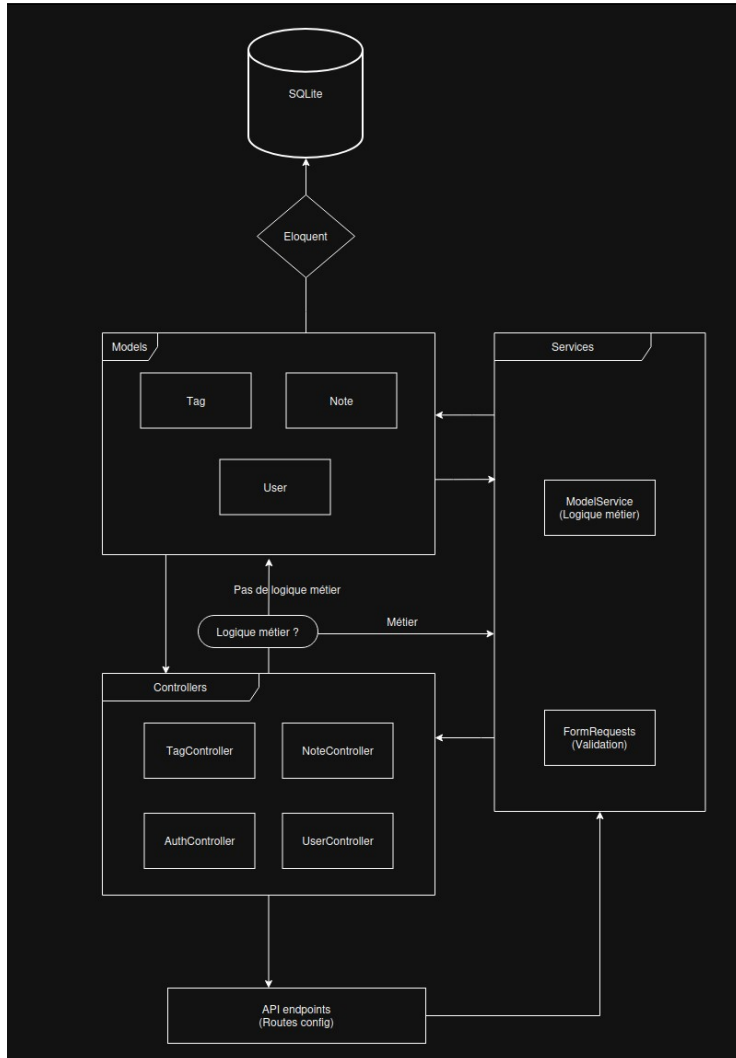
- **God components** : Notes.php mélange 5 responsabilités. (UI, validation, autorisation, accès données, rendu)
- Duplication de la logique d'autorisation (Auth::id()) répété partout).

Ces défauts vont devenir de vrai problèmes quand l'app grossira et devra s'ouvrir à d'autres clients.

3. Architecture cible

a. Composants d'architecture

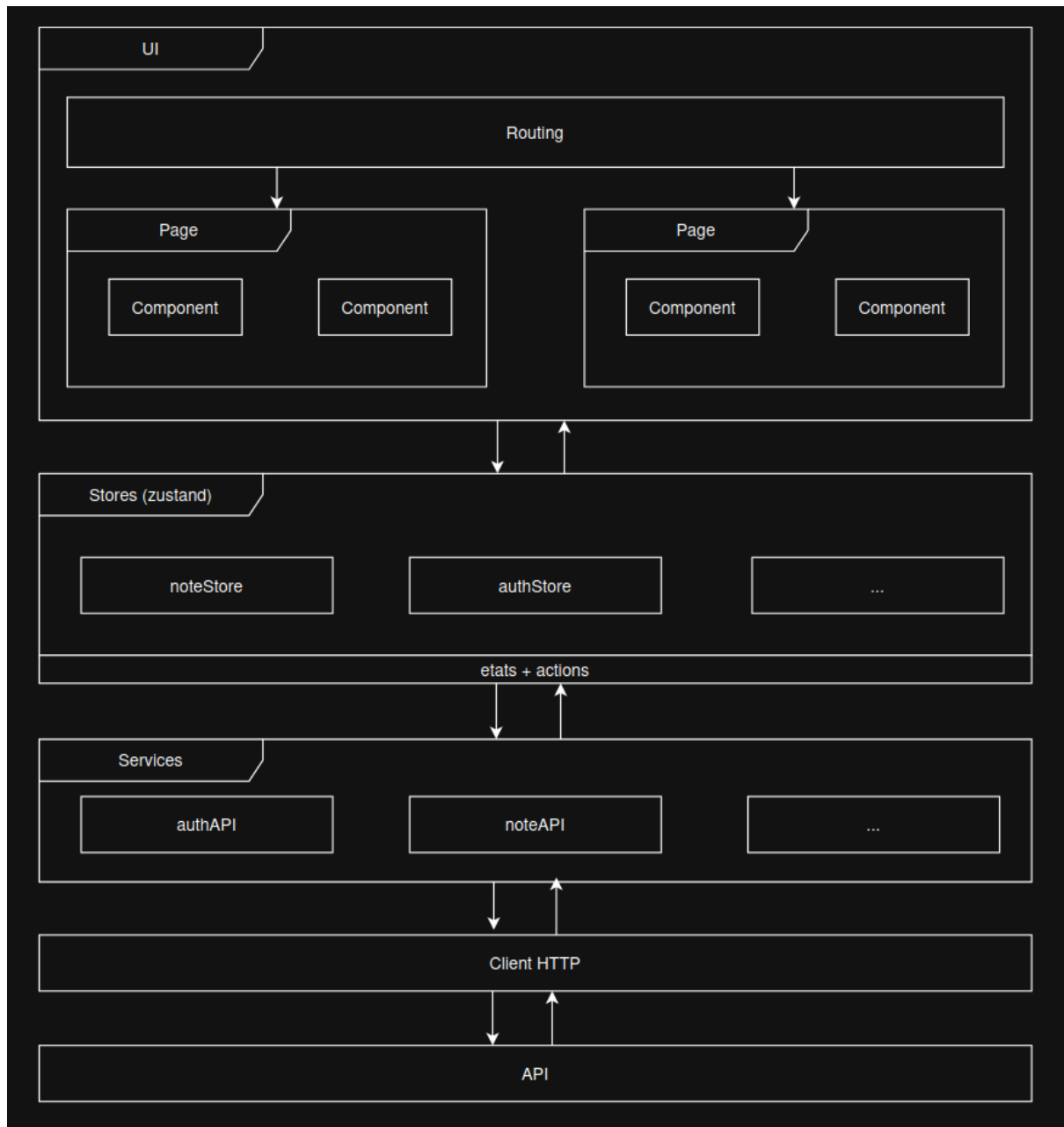
Back-end :



On conserve les models, qui communiquent grâce à eloquent avec la BDD en SQLite, mais on remplace le reste :

- On ajoute des services qui vont processor la logique métier (generation d'un token, valider une RQT pour que la donnée soit cohérente en back)
- On ajoute des controllers appelés par chaque route, qui font appels aux services pour la logique métier,
- Pour l'accès aux données, passage par les models, et eloquent
- On configure des routes (sur le modèle d'API REST) pour pouvoir requetter les controllers

Front-end :



Création d'une toute nouvelle application en react.

- L'utilisateur naviguera entre la pages grâce a la configuration des routes.
- Les pages seront constitués de composants qui se chargerons uniquement de l'affichage,
- La gestion des données sera géré dans les stores
- La communication avec le backend se fera via des services qui passerons tous via un client HTTP commun pour communiquer avec l'API

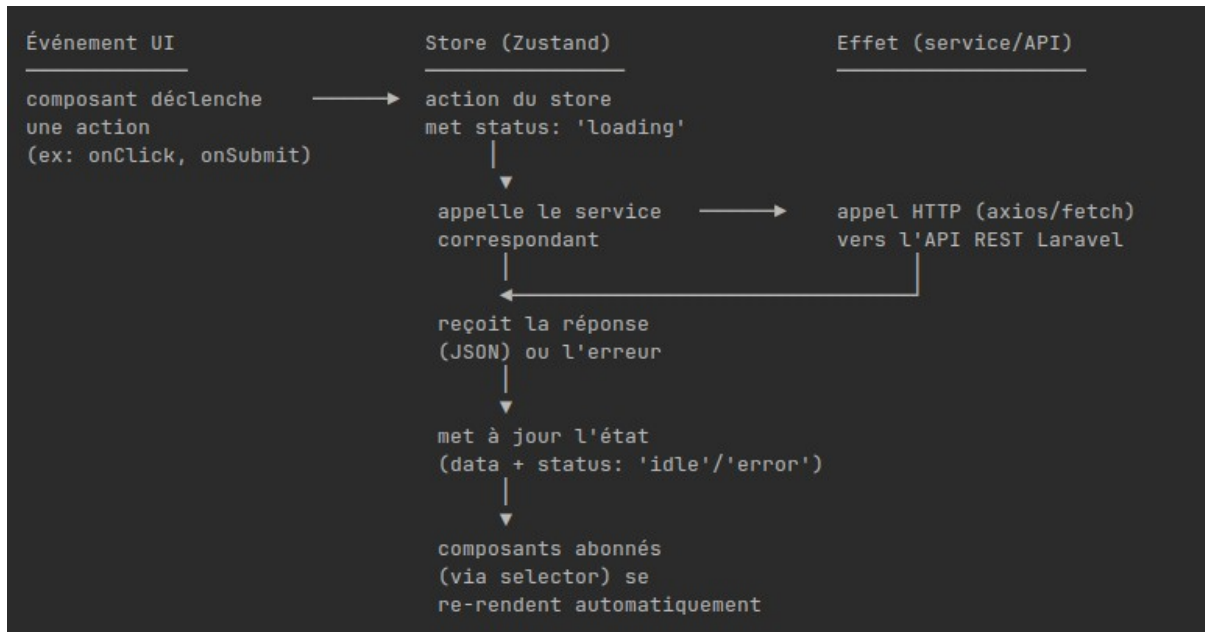
Organisation des fichiers :

```
src/
├── app/
│   ├── App.jsx           // composant racine, monte le routeur et les providers globaux
│   └── router.jsx        // déclaration des routes client (/login, /dashboard, /settings/...)
├── features/
│   ├── notes/
│   │   ├── components/
│   │   │   ├── NoteForm.jsx
│   │   │   ├── NoteList.jsx
│   │   │   └── NoteItem.jsx
│   │   ├── store/
│   │   │   └── notesStore.js
│   │   └── api/
│   │       └── notesApi.js
│   ├── tags/
│   │   ├── components/
│   │   │   ├── TagForm.jsx
│   │   │   ├── TagList.jsx
│   │   │   └── TagBadge.jsx
│   │   ├── store/
│   │   │   └── tagsStore.js
│   │   └── api/
│   │       └── tagsApi.js
│   ├── auth/
│   │   ├── components/
│   │   │   ├── LoginForm.jsx
│   │   │   ├── RegisterForm.jsx
│   │   │   ├── ForgotPasswordForm.jsx
│   │   │   ├── ResetPasswordForm.jsx
│   │   │   └── ConfirmPasswordForm.jsx
│   │   ├── store/
│   │   │   └── authStore.js // utilisateur courant, token, statut de connexion
│   │   └── api/
│   │       └── authApi.js
│   └── settings/
│       ├── components/
│       │   ├── ProfileForm.jsx
│       │   ├── PasswordForm.jsx
│       │   ├── AppearanceForm.jsx
│       │   └── DeleteAccountForm.jsx
│       ├── store/
│       │   └── settingsStore.js // ou réutilisation d'authStore pour le profil
│       └── api/
│           └── settingsApi.js
├── shared/
│   ├── components/
│   │   ├── Layout.jsx           // équivalent des layouts Blade actuels
│   │   ├── Sidebar.jsx
│   │   ├── Header.jsx
│   │   └── AppLogo.jsx
│   └── lib/
│       └── httpClient.js        // instance axios : base URL, intercepteur d'authentification, gestion centralisée des erreurs 401/422
└── main.jsx                   // point d'entrée de l'application
```

b. Spécifications API REST

Voir **docs/api-specs.md** dans le projet back-end

c. Data flow du state management



Principes de la séparation :

- UI (composants) : déclenchent des actions, ne contiennent aucune logique métier ni appel réseau.
- Store (Zustand) : détient l'état (données + statut idle/loading/error) et orchestre la séquence ; c'est le seul endroit qui appelle les services.
- Effets (couche services/*Api.ts) : seul point d'accès réseau de l'application, un module par ressource (notesApi, tagsApi, authApi...), aucun composant n'appelle fetch/axios directement.
- UI ← Store : la mise à jour de l'état déclenche automatiquement le re-rendu des composants abonnés (via un selector, une fonction qui lit une portion précise de l'état), pas de rafraîchissement manuel.

4. Analyse de l'écart

A ajouter :

- Contrôleurs : Partie Auth avec les controllers pour l'enregistrement et la connexion, et partie CRUD pour les Tags et Notes
- Services : Externaliser les logiques métier dans des services
- Routes : Créer les routes pour les différents endpoints API
- Création d'une toute nouvelle application react pour la partie frontend

A supprimer :

- Components livewire : comportent trop de responsabilités différents au sein d'un même fichier
- Templates Blade : Passage à un fonctionnement où le front et back sont séparés, donc les vues seront gérées par react en front.

5. Justification de l'approche architecturale

Zustand pour la gestion d'état

Zustand a été préféré à Redux et au Context API de React. Zustand permet d'écrire des actions sous forme de fonctions simples tout en ne déclenchant le re-rendu que des composants réellement concernés par la portion d'état modifiée.

Séparation entre interface, état et appels réseau (front)

Le front est structuré en trois couches distinctes : les composants, responsables uniquement de l'affichage ; le store, qui porte l'état applicatif ; et les services, qui centralisent les appels à l'API. Cette séparation limite l'impact d'une évolution technique : un changement de librairie de state management ou de mode d'appel à l'API n'affecte que la couche concernée, sans répercussion sur les composants. Elle facilite également les tests, la logique du store pouvant être vérifiée indépendamment d'un appel réseau réel.

Architecture MVC et principes SOLID (back)

L'architecture actuelle, portée par les composants Livewire, mélange affichage, validation et accès aux données au sein d'un même fichier, ce qui rend toute modification risquée pour les responsabilités adjacentes. La réorganisation autour de contrôleurs dédiés à l'orchestration, de services portant la logique métier et de modèles limités à l'accès aux données répond au principe de responsabilité unique issu de SOLID : chaque couche n'a qu'une seule raison d'évoluer. Une règle métier peut ainsi être modifiée sans impacter le contrôleur, et inversement.

Organisation du code par domaine

Le code est regroupé par domaine métier (notes, tags, authentication) plutôt que par nature technique, aussi bien côté front que côté back. Cette organisation modulaire facilite la localisation et l'évolution du code : l'ensemble des éléments relatifs à une fonctionnalité se trouve regroupé, plutôt que réparti entre plusieurs répertoires organisés par type de fichier.

Stabilité du contrat d'API

L'uniformisation du format de réponse (status, message, data) et des codes HTTP garantit qu'un client, qu'il s'agisse du front React actuel ou d'une future application mobile, peut consommer l'API sans connaître les détails de son implémentation. Ce découplage est la condition nécessaire pour qu'une même API serve plusieurs clients sans duplication de logique.

Décisions liées à la sécurité

L'authentification bascule d'un mécanisme par session/cookie vers un mécanisme par token, ce dernier étant plus adapté à une consommation par un client mobile, pour lequel la gestion de cookies de session est peu praticable. La validation des données est maintenue côté back indépendamment de celle réalisée côté front, ce dernier ne pouvant constituer une garantie suffisante puisque l'API reste accessible directement, hors de l'interface. Enfin, l'accès à une ressource n'appartenant pas à l'utilisateur authentifié renvoie une réponse 404 plutôt que 403, afin de ne pas révéler l'existence de données appartenant à un autre utilisateur.